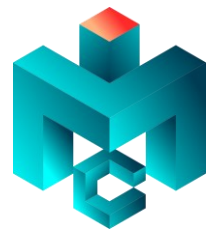




UNIVERSIDADE FEDERAL DE ITAJUBÁ
Instituto de Matemática e Computação



CMAC03 – Algoritmos em Grafos

2. Conceitos de Teoria dos Grafos

Busca

Rafael Frinhani

frinhani@unifei.edu.br

2026

Apresentar as definições de busca em grafos, os algoritmos de busca em largura e busca em profundidade e suas aplicações.

AGENDA

2. Teoria dos Grafos

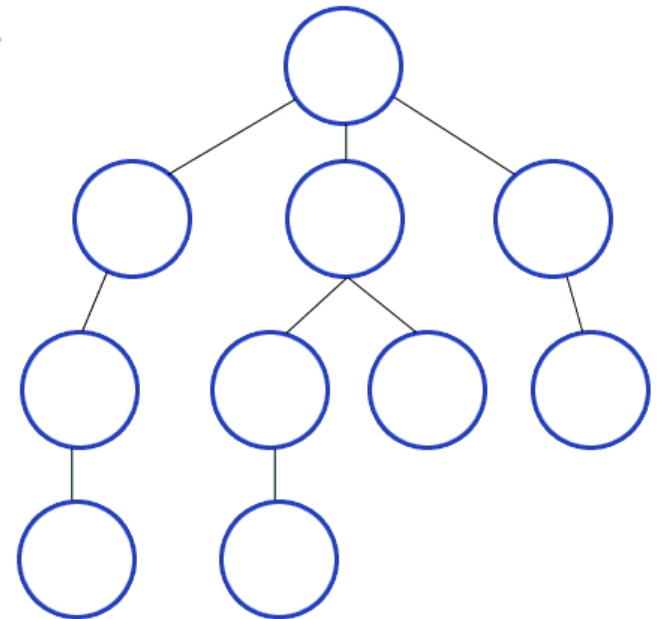
2.7. Busca em grafos

Contextualização, Aplicações, Algoritmo genérico de busca.

Busca em Largura (*Breadth First Search*, BFS), conceitos, algoritmo, níveis da busca.

Busca em Profundidade (*Deep First Search*, DFS), conceitos, algoritmo.

Análise de Complexidade algoritmos BFS e DFS.





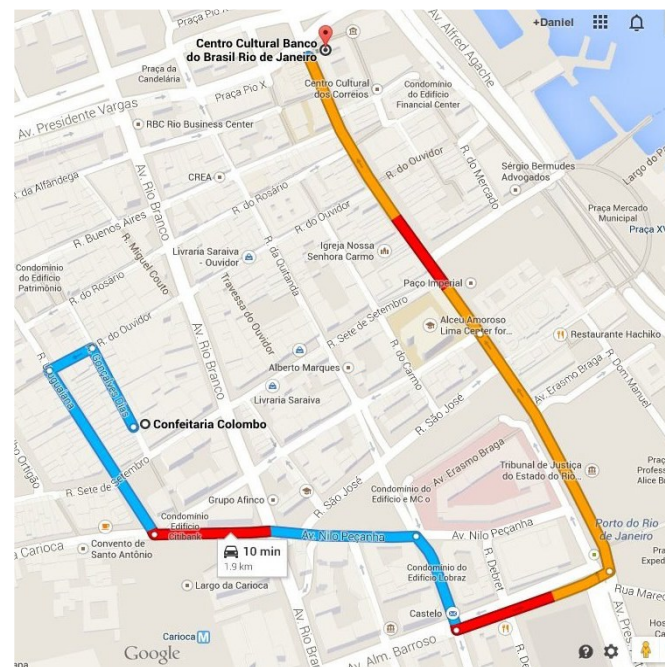
2.7. Busca em Grafos

Contextualização

A busca em grafos, ou percurso em grafos, é a tarefa de explorar ou percorrer de forma sistemática seus vértices e arestas para examiná-los.

A exploração pode ser utilizada para diversos propósitos como:

- encontrar um elemento (busca)
- identificar estruturas e propriedades
- descobrir caminhos
- base para métodos mais avançados
- resolver problemas modelados em grafos etc.





2.7. Busca em Grafos

Contextualização

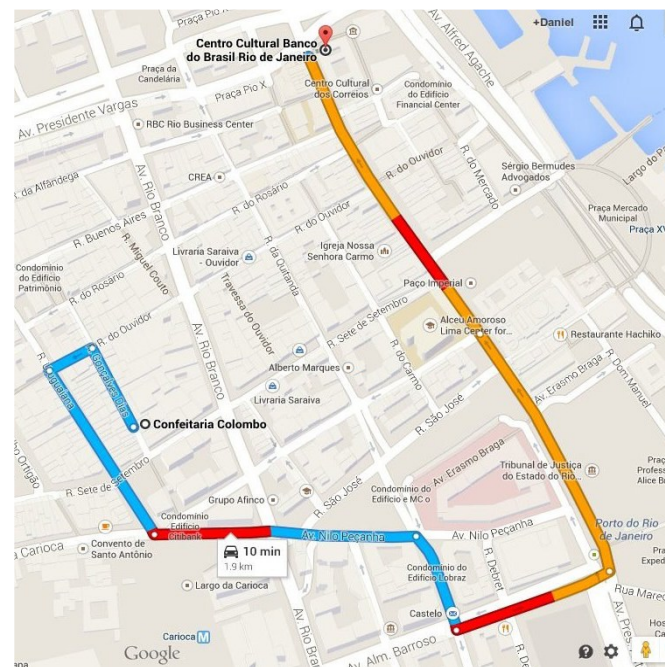
A busca em grafos, ou percurso em grafos, é a tarefa de explorar ou percorrer de forma sistemática seus vértices e arestas para examiná-los.

A exploração pode ser utilizada para diversos propósitos como:

- encontrar um elemento (busca)
- identificar estruturas e propriedades
- descobrir caminhos
- base para métodos mais avançados
- resolver problemas modelados em grafos etc.

A busca em grafo é **análoga a tarefa de encontrar um caminho em um labirinto** que consiste de passagens (arestas) e suas interseções (vértices).

Pode ser mais fácil encontrar a saída via um procedimento sistemático do que aleatoriamente.





2.7. Busca em Grafos

Algoritmos de Busca – Aplicações

- Determinar se um grafo é conexo
- Calcular distância entre dois vértices (comprimento do menor caminho)
- Determinar um caminho (problema do labirinto), ou caminho mínimo
- Detectar se o grafo possui ciclos
- Encontrar componentes bi conexas, testar se um grafo é bipartido
- Classificar arestas
- Encontrar componentes fortemente conexas
- Obter uma árvore geradora mínima
- *Flood Fill* (inundação)
- Ordenação Topológica
- Casos que requerem que seja retornada uma lista com a ordem em que os vértices foram visitados.



2.7. Busca em Grafos

Busca Genérica em Grafos

Algoritmos de busca evitam revisitar vértices já explorados utilizando uma marcação:

Não-Visitado: desconhecido ou não-explorado (*white*) corresponde aos vértices *ainda não descobertos* pelo algoritmo.

Visitado: ou descoberto (*gray*) são os vértices *visitados*, mas cujos *adjacentes ainda não foram analisados*.

Analisado: ou explorado (*black*) são os vértices cujos *adjacentes foram todos visitados*.



2.7. Busca em Grafos

Busca Genérica em Grafos

Algoritmos de busca evitam revisitar vértices já explorados utilizando uma marcação:

Não-Visitado: desconhecido ou não-explorado (*white*) corresponde aos vértices ainda não descobertos pelo algoritmo.

Visitado: ou descoberto (*gray*) são os vértices visitados, mas cujos adjacentes ainda não foram analisados.

Analisado: ou explorado (*black*) são os vértices cujos adjacentes foram todos visitados.

Algoritmo Genérico de Busca

Passo Inicial

- marcar todos os vértices como não-visitados
- selecionar vértice origem e marcá-lo como visitado

Passo geral

- Enquanto existir vértice visitado
 - Selecionar algum vértice visitado v_i
 - Se v_i possuir adjacente não-visitado v_j
 - marcar v_j como visitado
 - Senão, marcar v_i como analisado.



2.7. Busca em Grafos

Busca Genérica em Grafos

Algoritmos de busca evitam revisitar vértices já explorados utilizando uma marcação:

Não-Visitado: desconhecido ou não-explorado (*white*) corresponde aos vértices ainda não descobertos pelo algoritmo.

Visitado: ou descoberto (*gray*) são os vértices visitados, mas cujos adjacentes ainda não foram analisados.

Analisado: ou explorado (*black*) são os vértices cujos adjacentes foram todos visitados.

Algoritmo Genérico de Busca

Passo Inicial

marcar todos os vértices como não-visitados
selecionar vértice origem e marcá-lo como visitado

Passo geral

Enquanto existir vértice visitado
 Selecionar algum vértice visitado v_i
 Se v_i possuir adjacente não-visitado v_j
 marcar v_j como visitado
 Senão, marcar v_i como analisado.

Problema: algoritmo genérico não estabelece uma ordem de análise, ou seja, não define qual vértice v_j (visitado) deve ser escolhido para ser analisado.

A ordem de análise define uma sistemática de exploração. Técnicas fundamentais:

- v_j é o vértice visitado mais “antigo” (BFS)
- v_j é o vértice visitado mais “recente” (DFS)



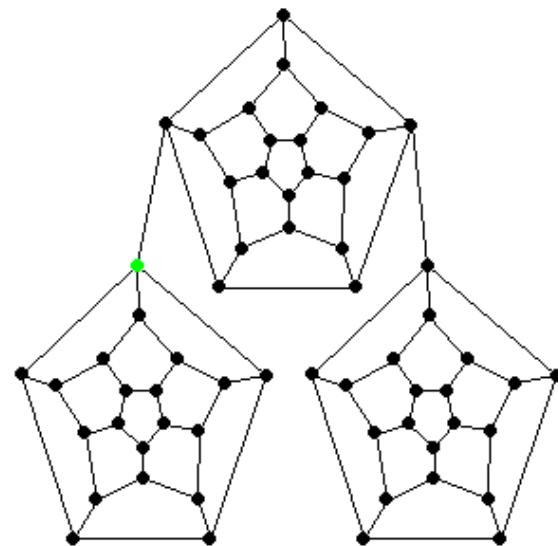
2.7. Busca em Grafos

Busca em Largura

Breadth First Search (BFS) ou Busca em Nível, a **exploração dos vértices progride na largura das conexões**, prioriza-se a análise de todos os vizinhos de um vértice origem.

BFS expande de maneira uniforme a fronteira (nível) entre vértices descobertos e não-descobertos. Um **vértice** é marcado como “analisado” ou “explorado” quando **todos os seus adjacentes forem visitados**.

Breadth-First Search



www.combinatorica.com



2.7. Busca em Grafos

Busca em Largura

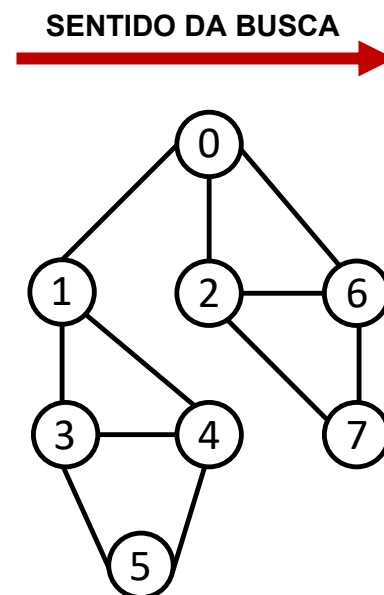
Breadth First Search (BFS) ou Busca em Nível, a exploração dos vértices progride na largura das conexões, prioriza-se a análise de todos os vizinhos de um vértice origem.

BFS expande de maneira uniforme a fronteira (nível) entre vértices descobertos e não-descobertos. Um vértice é marcado como “analisado” ou “explorado” quando todos os seus adjacentes forem visitados.

O algoritmo utiliza uma fila para o controle da ordem de análise dos vértices.

São analisados os vértices que foram visitados primeiro (mais antigos, adicionados na fila à mais tempo). Os vértices são visitados na ordem crescente de seus identificadores.

Grafos podem ser não-direcionados ou dígrafos.





2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

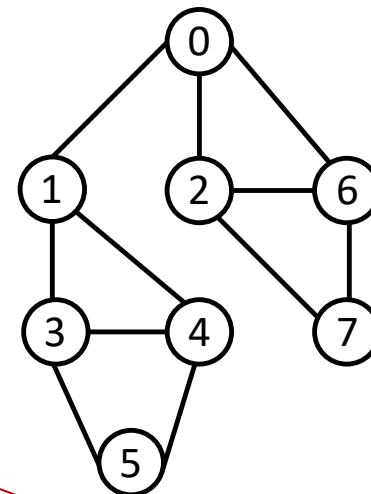
BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

► 0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

Inicialmente todos os
vértices são considerados
não-visitados (*white*)

Q corresponde a fila (*queue*)
dos vértices visitados (*gray*),
mas cujos vizinhos ainda não
foram inspecionados.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]



2.7. Busca em Grafos – Busca em Largura

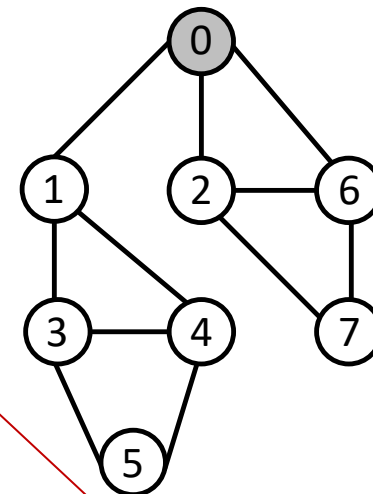
Algoritmo – BFS

BFS (G, v)

```
1  $Q = []$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

► 0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

Considerando o **vértice de índice 0 como inicial**, ele é inserido em Q .



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]



2.7. Busca em Grafos – Busca em Largura

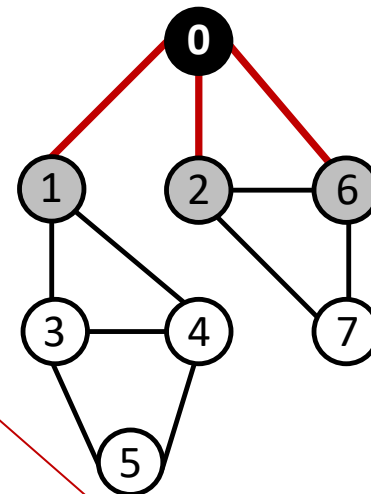
Algoritmo – BFS

BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

► 0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

Vértice 0 é removido da fila
e cada um dos seus
adjacentes ainda não
pertencentes a Q são
inseridos a esta fila pela
ordem crescente do índice.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]

Depois que todos os adjacentes do vértice foram
inseridos a Q ele é considerado analisado (preto).



2.7. Busca em Grafos – Busca em Largura

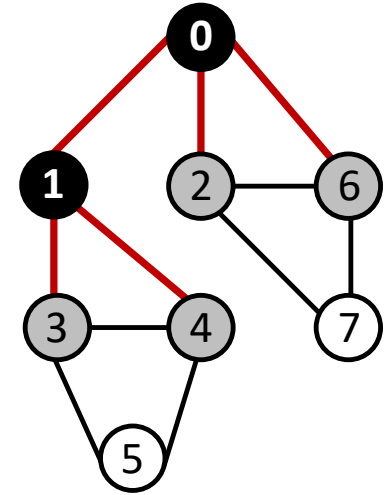
Algoritmo – BFS

BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

0 : [1, 2, 6]
▶ 1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

A sequência da busca continua com o **vértice 1**, que é o primeiro adjacente do vértice 0 que foi inserido à fila Q .



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2 , 3, 4, 5, 6 , 7	[0]
2	0	3, 4 , 5, 7	[<u>1</u> , 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]

Os **adjacentes do vértice 1** ainda não pertencentes Q a **são inseridos a fila** (vértices 3 e 4). Depois das inserções o vértice 1 é considerado analisado.



2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

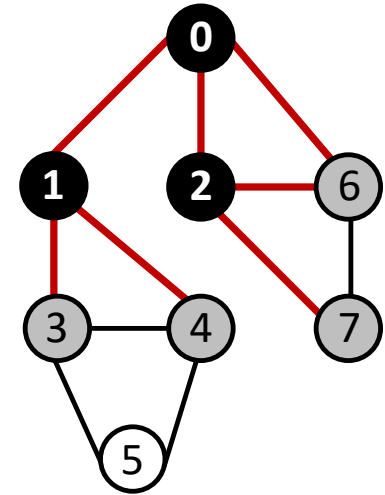
BFS (G, v)

```

1  $Q = [ ]$ ;
2 insere  $v$  em  $Q$ ;
3 while  $Q \neq \emptyset$  do
4    $v =$  remove o primeiro elemento de  $Q$ ;
5   for cada adjacente de  $v$  do
6     if adjacente  $\notin Q$  then
7       insere adjacente em  $Q$ ;
8     end
9   end
10  marca  $v$  como analisado;
11 end
  
```

0 : [1, 2, 6]
 1 : [0, 3, 4]
 ► 2 : [0, 6, 7]
 3 : [1, 4, 5]
 4 : [1, 3, 5]
 5 : [3, 4]
 6 : [0, 2, 7]
 7 : [2, 6]

O progresso da busca considera cada vértice no início da fila Q e a inserção dos seus adjacentes nesta fila quando necessário.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]
4	0, 1, 2	5	[6, 3, 4, 7]



2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

BFS (G, v)

```

1  Q = [ ];
2  insere v em Q;
3  while Q ≠ ∅ do
4      v = remove o primeiro elemento de Q;
5      for cada adjacente de v do
6          if adjacente ∉ Q then
7              insere adjacente em Q;
8          end
9      end
10     marca v como analisado;
11 end
  
```

0 : [1, 2, 6]

1 : [0, 3, 4]

2 : [0, 6, 7]

3 : [1, 4, 5]

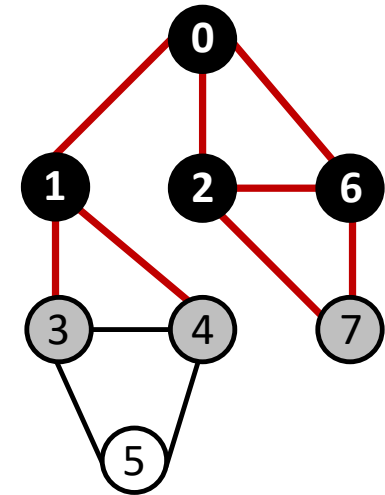
4 : [1, 3, 5]

5 : [3, 4]

▶ 6 : [0, 2, 7]

7 : [2, 6]

O progresso da busca considera cada vértice no início da fila *Q* e a inserção dos seus adjacentes nesta fila quando necessário.



Passo	Analisado	Não-Visitado	<i>Q</i> (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2 , 3, 4, 5, 6 , 7	[0]
2	0	3, 4 , 5, 7	[1 , 2, 6]
3	0, 1	5, 7	[2 , 6, 3, 4]
4	0, 1, 2	5	[6 , 3, 4, 7]
5	0, 1, 2, 6	5	[3 , 4, 7]



2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

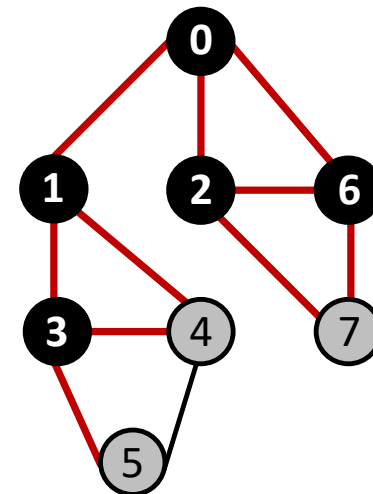
BFS (G, v)

```

1  $Q = [ ]$ ;
2 insere  $v$  em  $Q$ ;
3 while  $Q \neq \emptyset$  do
4    $v =$  remove o primeiro elemento de  $Q$ ;
5   for cada adjacente de  $v$  do
6     if adjacente  $\notin Q$  then
7       insere adjacente em  $Q$ ;
8     end
9   end
10  marca  $v$  como analisado;
11 end
  
```

0 : [1, 2, 6]
 1 : [0, 3, 4]
 2 : [0, 6, 7]
 ► 3 : [1, 4, 5]
 4 : [1, 3, 5]
 5 : [3, 4]
 6 : [0, 2, 7]
 7 : [2, 6]

O progresso da busca considera cada vértice no início da fila Q e a inserção dos seus adjacentes nesta fila quando necessário.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]
4	0, 1, 2	5	[6, 3, 4, 7]
5	0, 1, 2, 6	5	[3, 4, 7]
6	0, 1, 2, 6, 3	-	[4, 7, 5]



2.7. Busca em Grafos – Busca em Largura

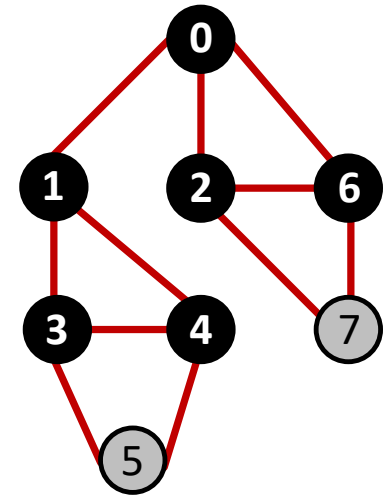
Algoritmo – BFS

BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
▶ 4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

O progresso da busca considera cada vértice no início da fila Q e a inserção dos seus adjacentes nesta fila quando necessário.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]
4	0, 1, 2	5	[6, 3, 4, 7]
5	0, 1, 2, 6	5	[3, 4, 7]
6	0, 1, 2, 6, 3	-	[4, 7, 5]
7	0, 1, 2, 6, 3, 4	-	[7, 5]



2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

0 : [1, 2, 6]

1 : [0, 3, 4]

2 : [0, 6, 7]

3 : [1, 4, 5]

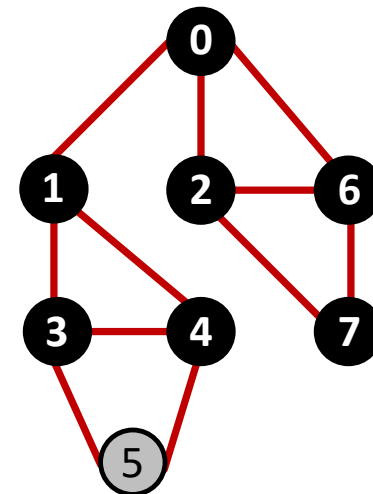
4 : [1, 3, 5]

5 : [3, 4]

6 : [0, 2, 7]

▶ 7 : [2, 6]

O progresso da busca considera cada vértice no início da fila Q e a inserção dos seus adjacentes nesta fila quando necessário.



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]
4	0, 1, 2	5	[6, 3, 4, 7]
5	0, 1, 2, 6	5	[3, 4, 7]
6	0, 1, 2, 6, 3	-	[4, 7, 5]
7	0, 1, 2, 6, 3, 4	-	[7, 5]
8	0, 1, 2, 6, 3, 4, 7	-	[5]



2.7. Busca em Grafos – Busca em Largura

Algoritmo – BFS

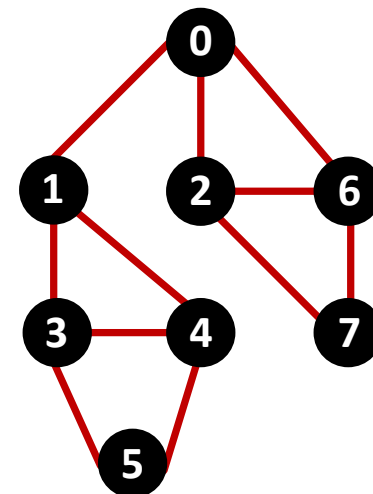
BFS (G, v)

```
1  $Q = [ ]$ ;  
2 insere  $v$  em  $Q$ ;  
3 while  $Q \neq \emptyset$  do  
4    $v =$  remove o primeiro elemento de  $Q$ ;  
5   for cada adjacente de  $v$  do  
6     if adjacente  $\notin Q$  then  
7       insere adjacente em  $Q$ ;  
8     end  
9   end  
10  marca  $v$  como analisado;  
11 end
```

0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
▶ 5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

A sequência de visitas dos
vértices na Busca em Largura
com o vértice 0 como origem é:

$BFS(0) = \{0, 1, 2, 6, 3, 4, 7, 5\}$



Passo	Analisado	Não-Visitado	Q (Visitado)
0	-	0, 1, 2, 3, 4, 5, 6, 7	[]
1	-	1, 2, 3, 4, 5, 6, 7	[0]
2	0	3, 4, 5, 7	[1, 2, 6]
3	0, 1	5, 7	[2, 6, 3, 4]
4	0, 1, 2	5	[6, 3, 4, 7]
5	0, 1, 2, 6	5	[3, 4, 7]
6	0, 1, 2, 6, 3	-	[4, 7, 5]
7	0, 1, 2, 6, 3, 4	-	[7, 5]
8	0, 1, 2, 6, 3, 4, 7	-	[5]
9	0, 1, 2, 6, 3, 4, 7, 5	-	[]

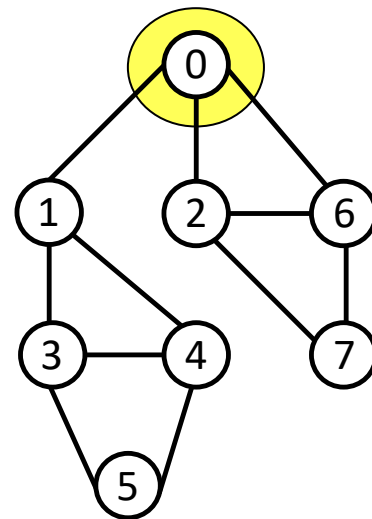


2.7. Busca em Grafos – Busca em Largura

Níveis da Busca

A progressão da BFS é análoga a de uma onda que é propagada a partir do vértice origem.

A onda expande em círculos que determinam níveis dos vizinhos a serem analisados em relação a origem.



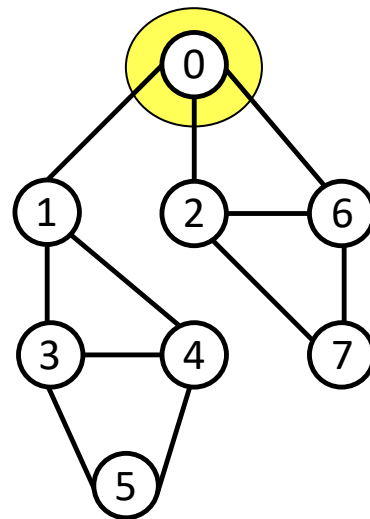


2.7. Busca em Grafos – Busca em Largura

Níveis da Busca

A progressão da BFS é análoga a de uma onda que é propagada a partir do vértice origem.

A onda expande em círculos que determinam níveis dos vizinhos a serem analisados em relação a origem.



NÍVEIS
 $L_0 : 0$

NÍVEIS (camadas)

- L_i : conjunto de vértices pertencentes ao nível
 $i = \{0, 1, 2, \dots, n\}$
- L_0 : vértice origem
- L_{i+1} : conjunto de vértices que fazem parte de um nível posterior e que possuem uma aresta com algum vértice do nível L_i

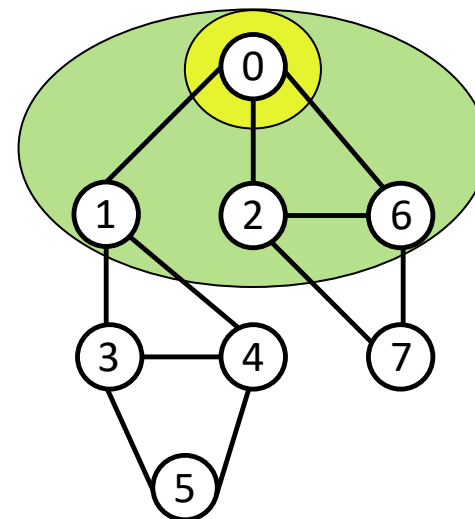


2.7. Busca em Grafos – Busca em Largura

Níveis da Busca

A progressão da BFS é análoga a de uma onda que é propagada a partir do vértice origem.

A onda expande em círculos que determinam níveis dos vizinhos a serem analisados em relação a origem.



NÍVEIS (camadas)

- L_i : conjunto de vértices pertencentes ao nível
 $i = \{0, 1, 2, \dots, n\}$
- L_0 : vértice origem
- L_{i+1} : conjunto de vértices que fazem parte de um nível posterior e que possuem uma aresta com algum vértice do nível L_i

NÍVEIS

$L_0 : 0$

$L_1 : 1, 2, 6$

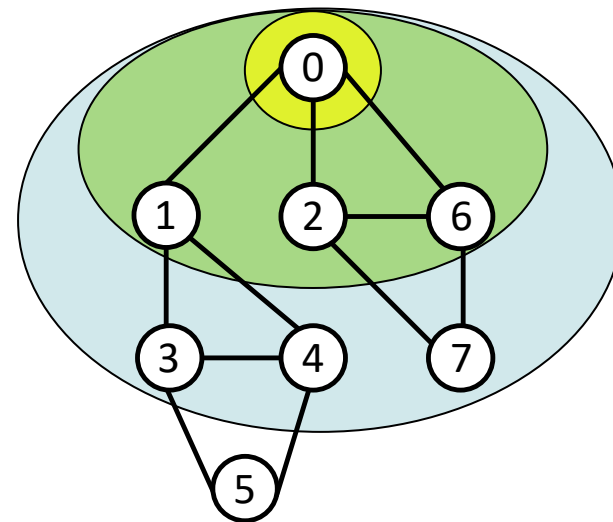


2.7. Busca em Grafos – Busca em Largura

Níveis da Busca

A progressão da BFS é análoga a de uma onda que é propagada a partir do vértice origem.

A onda expande em círculos que determinam níveis dos vizinhos a serem analisados em relação a origem.



NÍVEIS (camadas)

- L_i : conjunto de vértices pertencentes ao nível
 $i = \{0, 1, 2, \dots, n\}$
- L_0 : vértice origem
- L_{i+1} : conjunto de vértices que fazem parte de um nível posterior e que possuem uma aresta com algum vértice do nível L_i

NÍVEIS

L_0 : 0
 L_1 : 1, 2, 6
 L_2 : 3, 4, 7

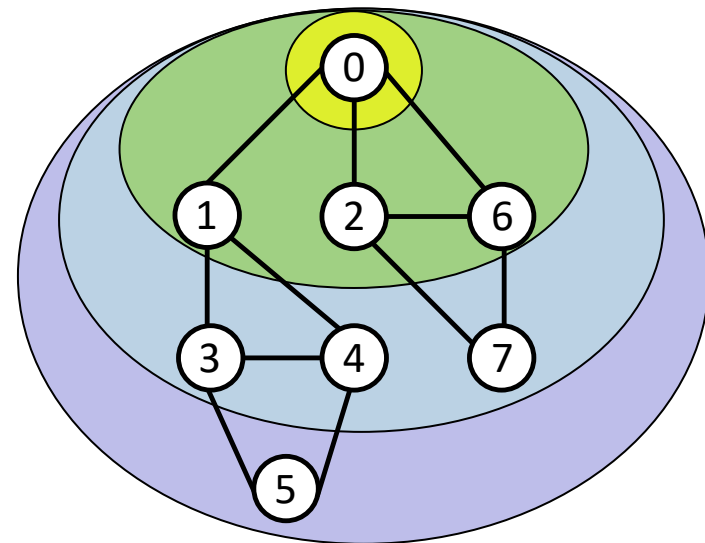


2.7. Busca em Grafos – Busca em Largura

Níveis da Busca

A progressão da BFS é análoga a de uma onda que é propagada a partir do vértice origem.

A onda expande em círculos que determinam níveis dos vizinhos a serem analisados em relação a origem.



NÍVEIS (camadas)

- L_i : conjunto de vértices pertencentes ao nível $i = \{0, 1, 2, \dots, n\}$
- L_0 : vértice origem
- L_{i+1} : conjunto de vértices que fazem parte de um nível posterior e que possuem uma aresta com algum vértice do nível L_i

	NÍVEIS
Distância é o	$L_0 : 0$
comprimento de	$L_1 : 1, 2, 6$
menor caminho entre	$L_2 : 3, 4, 7$
dois vértices.	$L_3 : 5$

Vértices pertencentes a camada L_i têm distância i da origem.



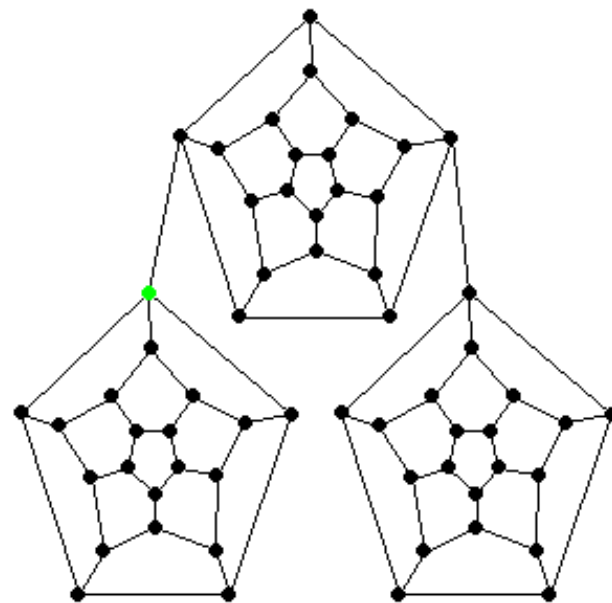
2.7. Busca em Grafos

Busca em Profundidade

Ou *Deep First Search* (DFS), a exploração dos vértices progride na profundidade (extensão) das conexões. A estratégia é buscar o mais profundo possível no grafo, voltando no percurso quando não existem mais vértices não-visitados pela frente.

A partir do vértice origem são explorados os adjacentes descobertos mais recentemente (mais novos). Não analisa todos os vértices de um nível antes de seguir para o próximo nível.

Depth-First Search



www.combinatorica.com



2.7. Busca em Grafos

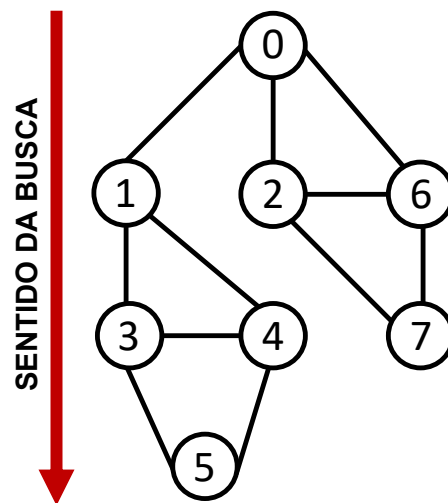
Busca em Profundidade

Ou Deep First Search (DFS), a exploração dos vértices progride na profundidade (extensão) das conexões. A estratégia é buscar o mais profundo no grafo possível, voltando no percurso quando não existem mais vértices não-visitados pela frente.

A partir do vértice origem são explorados os adjacentes descobertos mais recentemente (mais novos). Não analisa todos os vértices de um nível antes de seguir para o próximo nível.

Uma **pilha** (recursiva ou explícita) controla a ordem de análise dos vértices, que são visitados na ordem crescente de seus ids.

Na DFS um vértice é marcado como visitado antes que toda sua vizinhança seja visitada. Grafos podem ser não-direcionados ou dígrafos.





2.7. Busca em Grafos – Busca em Profundidade

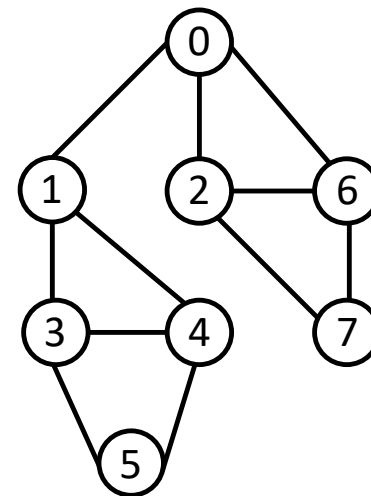
Algoritmo – DFS (Recursivo)

DFS (G, v)

```
1  marca  $v$  como visitado;  
2  for cada adjacente de  $v$  do  
3    if adjacente não-visitado then  
4      |  $DFS(G, adjacente)$ ;  
5    end  
6  end
```

0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

Inicialmente todos os vértices
são considerados não-
visitados (*white*)



Passo	Visitado	Não-Visitado	Pilha
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

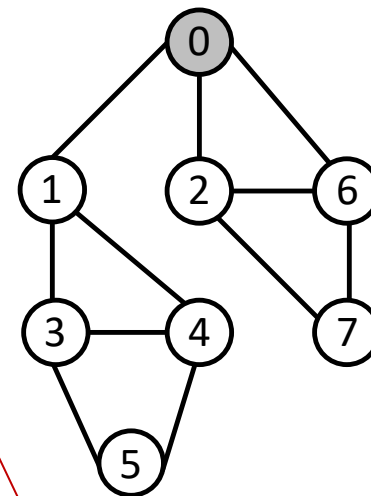
```
1 marca v como visitado;  
2 for cada adjacente de v do  
3   | if adjacente não-visitado then  
4   |   | DFS(G, adjacente);  
5   | end  
6 end
```

```
▶ 0 : [1, 2, 6]  
1 : [0, 3, 4]  
2 : [0, 6, 7]  
3 : [1, 4, 5]  
4 : [1, 3, 5]  
5 : [3, 4]  
6 : [0, 2, 7]  
7 : [2, 6]
```

Inicialmente todos os vértices
são considerados não-
visitados (*white*)

O algoritmo inicia empilhando
o vértice dado como origem
(para o caso o de índice 0).

Uma vez na pilha o vértice é
considerado visitado (*gray*).



Passo	Visitado	Não-Visitado	Pilha
1	0	1, 2, 3, 4, 5, 6, 7	DFS(0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (*G*, *v*)

```

1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          |   DFS(G, adjacente);
5      end
6  end

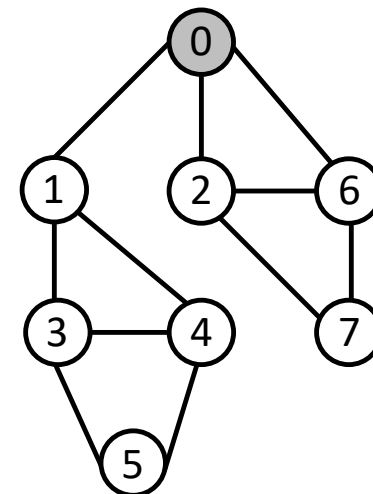
```

▶ 0 : [1, 2, 6]
 1 : [0, 3, 4]
 2 : [0, 6, 7]
 3 : [1, 4, 5]
 4 : [1, 3, 5]
 5 : [3, 4]
 6 : [0, 2, 7]
 7 : [2, 6]

Inicialmente todos os vértices
são considerados não-
visitados (*white*)

O algoritmo inicia empilhando
o vértice dado como origem
(para o caso o de índice 0).

Uma vez na pilha o vértice é
considerado visitado (*gray*).



Após o empilhamento do vértice 0 é selecionado um
adjacente não-visitado conforme a ordem crescente do
índice (**vértice 1**).

Passo	Visitado	Não-Visitado	Pilha
1	0	1, 2, 3, 4, 5, 6, 7	<i>DFS</i> (0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

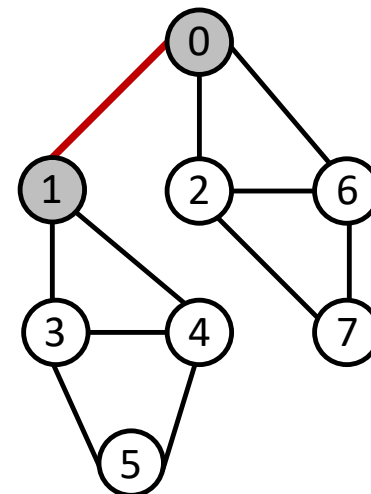
```

1  marca v como visitado;
2  for cada adjacente de v do
3    |  if adjacente não-visitado then
4    |  |  DFS(G, adjacente);
5    |  end
6  end

```

O vértice 1 é dado como entrada para a função recursiva (linha 4), sendo empilhado e marcado como visitado.

A busca continua com o vértice adjacente que ainda não foi visitado (vértice 3).



```

0 : [1, 2, 6]
1 : [0, 3, 4]
2 : [0, 6, 7]
3 : [1, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

```

Passo	Visitado	Não-Visitado	Pilha
2	0, 1	2, 3, 4, 5, 6, 7	DFS(1) = [0 , 3, 4]
1	0	1, 2, 3, 4, 5, 6, 7	DFS(0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

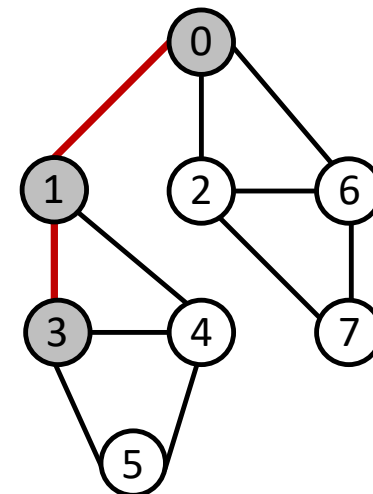
Algoritmo – DFS (Recursivo)

DFS (G, v)

```
1 marca v como visitado;  
2 for cada adjacente de v do  
3   if adjacente não-visitado then  
4     | DFS(G, adjacente);  
5   end  
6 end
```

O vértice 1 é dado como entrada para a função recursiva (linha 4), sendo empilhado e marcado como visitado.

A busca continua com o vértice adjacente que ainda não foi visitado (vértice 3).



0 : [1, 2, 6]
1 : [~~0~~, 3, 4]
2 : [0, 6, 7]
▶ 3 : [~~1~~, 4, 5]
4 : [1, 3, 5]
5 : [3, 4]
6 : [0, 2, 7]
7 : [2, 6]

Vértice 3 é dado como entrada para a função recursiva DFS, sendo selecionado o próximo adjacente disponível (vértice 4).

Passo	Visitado	Não-Visitado	Pilha
3	0, 1, 3	2, 4, 5, 6, 7	$DFS(3) = \textcolor{red}{1}, 4, 5 $
2	0, 1	2, 3, 4, 5, 6, 7	$DFS(1) = \textcolor{red}{0}, 3, 4 $
1	0	1, 2, 3, 4, 5, 6, 7	$DFS(0) = 1, 2, 6 $
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

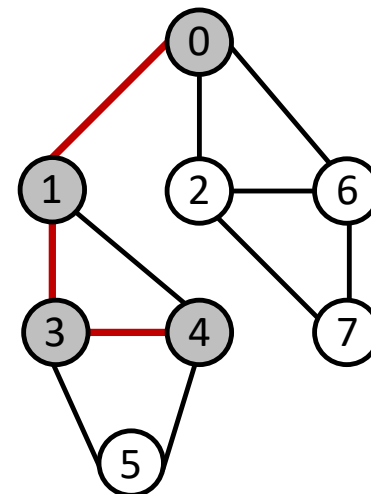
Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3    if adjacente não-visitado then
4      | DFS(G, adjacente);
5    end
6  end
    
```

A busca continua com o empilhamento do vértice mais recente enquanto existir adjacências ainda não-visitadas.



0 : [1, 2, 6]
 1 : [~~0~~, 3, 4]
 2 : [0, 6, 7]
 3 : [~~0~~, 4, 5]
 4 : [~~0~~, ~~1~~, 5]
 5 : [3, 4]
 6 : [0, 2, 7]
 7 : [2, 6]

Passo	Visitado	Não-Visitado	Pilha
4	0, 1, 3, 4	2, 5, 6, 7	DFS(4) = [1 , 3, 5]
3	0, 1, 3	2, 4, 5, 6, 7	DFS(3) = [1 , 4, 5]
2	0, 1	2, 3, 4, 5, 6, 7	DFS(1) = [0 , 3, 4]
1	0	1, 2, 3, 4, 5, 6, 7	DFS(0) = [1, 2, 6]
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

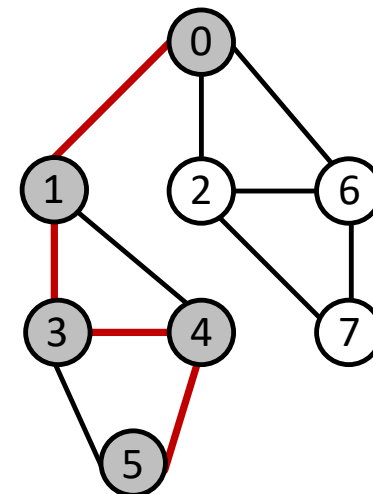
DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3    if adjacente não-visitado then
4      DFS(G, adjacente);
5    end
6  end
    
```

A busca continua com o empilhamento do vértice mais recente enquanto existir adjacências ainda não-visitadas.

No vértice 5 não existem mais adjacências não-visitadas, iniciando o processo de **desempilhamento** (*backtracking*).



0 : [1, 2, 6]
 1 : [~~0~~, 3, 4]
 2 : [0, 6, 7]
 3 : [~~0~~, 4, 5]
 4 : [~~0~~, ~~1~~, 5]
 ► 5 : [~~0~~, ~~1~~]
 6 : [0, 2, 7]
 7 : [2, 6]

Passo	Visitado	Não-Visitado	Pilha
5	0, 1, 3, 4, 5	2, 6, 7	$DFS(5) = \textcolor{red}{3}, \textcolor{red}{4} $
4	0, 1, 3, 4	2, 5, 6, 7	$DFS(4) = \textcolor{red}{1}, 3, 5 $
3	0, 1, 3	2, 4, 5, 6, 7	$DFS(3) = \textcolor{red}{1}, 4, 5 $
2	0, 1	2, 3, 4, 5, 6, 7	$DFS(1) = \textcolor{red}{0}, 3, 4 $
1	0	1, 2, 3, 4, 5, 6, 7	$DFS(0) = \textcolor{red}{1}, 2, 6 $
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

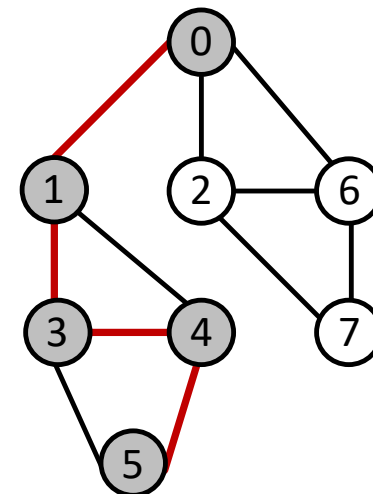
1  marca v como visitado;
2  for cada adjacente de v do
3    if adjacente não-visitado then
4      | DFS(G, adjacente);
5    end
6  end
    
```

0 : [1, 2, 6]
 1 : [~~0~~, 3, 4]
 2 : [0, 6, 7]
 3 : [~~0~~, 4, 5]
 ► 4 : [~~0~~, ~~3~~, ~~5~~]
 5 : [~~0~~, ~~3~~]
 6 : [0, 2, 7]
 7 : [2, 6]

A busca continua com o empilhamento do vértice mais recente enquanto existir adjacências ainda não-visitadas.

No vértice 5 não existem mais adjacências não-visitadas, iniciando o processo de desempilhamento (*backtracking*).

Os vértices 4, 3 e 1 (desempilhados nesta ordem) já tem todos seus adjacentes visitados.



Passo	Visitado	Não-Visitado	Pilha
6	0, 1, 3, 4, 5	2, 6, 7	DFS(4) = [1 , 3, 5]
3	0, 1, 3	2, 4, 5, 6, 7	DFS(3) = [1 , 4, 5]
2	0, 1	2, 3, 4, 5, 6, 7	DFS(1) = [0, 3, 4]
1	0	1, 2, 3, 4, 5, 6, 7	DFS(0) = [1, 2, 6]
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

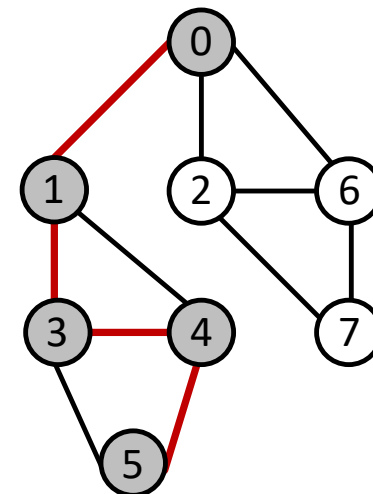
1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          | DFS(G, adjacente);
5      end
6  end
    
```

0 : [1, 2, 6]
 1 : [~~0~~, 3, 4]
 2 : [0, 6, 7]
 ► 3 : [~~0~~, ~~1~~, ~~6~~]
 4 : [~~0~~, ~~1~~, ~~6~~]
 5 : [~~0~~, ~~1~~]
 6 : [0, 2, 7]
 7 : [2, 6]

A busca continua com o empilhamento do vértice mais recente enquanto existir adjacentes ainda não-visitados.

No vértice 5 não existem mais adjacentes não-visitados, iniciando o processo de desempilhamento (*backtracking*).

Os vértices 4, 3 e 1 (desempilhados nesta ordem) já tem todos seus adjacentes visitados.



Passo	Visitado	Não-Visitado	Pilha
7	0, 1, 3, 4, 5	2, 6, 7	$DFS(3) = [1, 4, 5]$
2	0, 1	2, 3, 4, 5, 6, 7	$DFS(1) = [0, 3, 4]$
1	0	1, 2, 3, 4, 5, 6, 7	$DFS(0) = [1, 2, 6]$
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

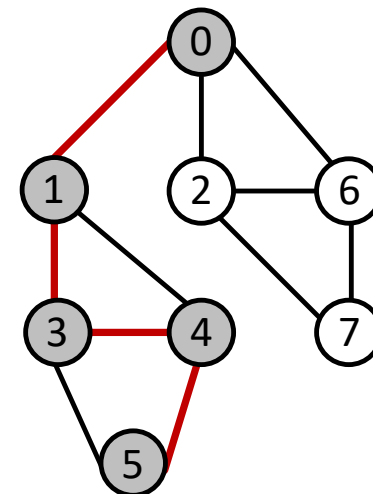
1  marca v como visitado;
2  for cada adjacente de v do
3    if adjacente não-visitado then
4      | DFS(G, adjacente);
5    end
6  end
    
```

0 : [1, 2, 6]
 ► 1 : [~~0~~, ~~2~~, ~~6~~]
 2 : [0, 6, 7]
 3 : [~~0~~, ~~2~~, ~~6~~]
 4 : [~~0~~, ~~2~~, ~~6~~]
 5 : [~~0~~, ~~2~~]
 6 : [0, 2, 7]
 7 : [2, 6]

A busca continua com o empilhamento do vértice mais recente enquanto existir adjacentes ainda não-visitados.

No vértice 5 não existem mais adjacentes não-visitados, iniciando o processo de desempilhamento (*backtracking*).

Os vértices 4, 3 e 1 (desempilhados nesta ordem) já tem todos seus adjacentes visitados.



Passo	Visitado	Não-Visitado	Pilha
8	0, 1, 3, 4, 5	2, 6, 7	$DFS(1) = [0, 3, 4]$
1	0	1, 2, 3, 4, 5, 6, 7	$DFS(0) = [1, 2, 6]$
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

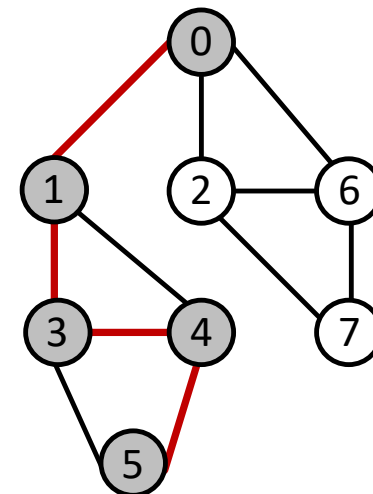
Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          | DFS(G, adjacente);
5      end
6  end
    
```

Quando a busca retorna a origem (vértice 0) é identificado um adjacente ainda não-visitado (vértice 2).



▶ 0 : [~~1~~, 2, 6]
 1 : [~~0~~, ~~3~~, ~~4~~]
 2 : [0, 6, 7]
 3 : [~~0~~, ~~1~~, ~~4~~]
 4 : [~~0~~, ~~1~~, ~~3~~]
 5 : [~~0~~, ~~3~~]
 6 : [0, 2, 7]
 7 : [2, 6]

Passo	Visitado	Não-Visitado	Pilha
9	0, 1, 3, 4, 5	2, 6, 7	DFS(0) = [1 , 2 , 6]
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

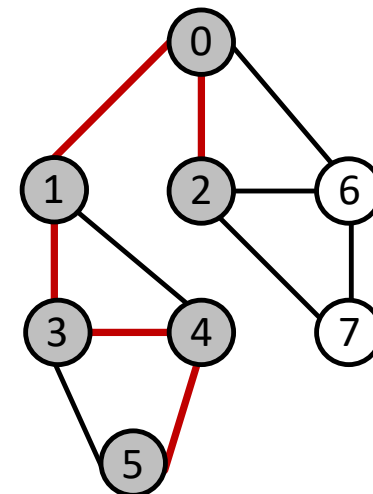
```

1  marca v como visitado;
2  for cada adjacente de v do
3    if adjacente não-visitado then
4      | DFS(G, adjacente);
5    end
6  end
    
```

0 : [~~1~~, 2, 6]
 1 : [~~0~~, ~~2~~, ~~6~~]
 2 : [0, 6, 7]
 3 : [~~0~~, ~~1~~, ~~6~~]
 4 : [~~0~~, ~~2~~, ~~6~~]
 5 : [~~0~~, ~~1~~]
 6 : [0, 2, 7]
 7 : [2, 6]

Quando a busca retorna a origem (vértice 0) é identificado um adjacente ainda não-visitado (vértice 2).

O processo de empilhamento recomeça a partir do vértice 2.



Passo	Visitado	Não-Visitado	Pilha
10	0, 1, 3, 4, 5, 2	6, 7	DFS(2) = 0, 6, 7
9	0, 1, 3, 4, 5	1, 2, 3, 4, 5, 6, 7	DFS(0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

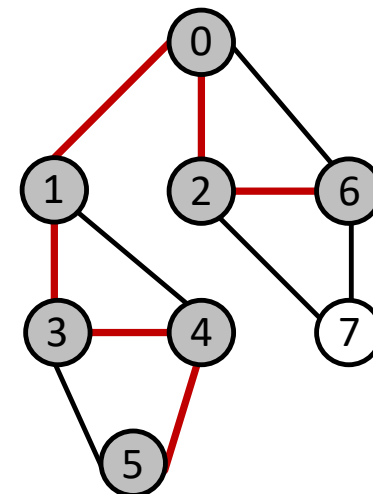
1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          | DFS(G, adjacente);
5      end
6  end
    
```

0 : [~~1~~, 2, 6]
 1 : [~~0~~, ~~2~~, ~~3~~]
 2 : [~~0~~, ~~1~~, ~~3~~]
 3 : [~~0~~, ~~1~~, ~~2~~]
 4 : [~~0~~, ~~1~~, ~~3~~]
 5 : [~~0~~, ~~4~~]
 6 : [~~0~~, ~~2~~, 7]
 7 : [2, 6]

Quando a busca retorna a origem (vértice 0) é identificado um adjacente ainda não-visitado (vértice 2).

O processo de empilhamento recomeça a partir do vértice 2.

Continua ao empilhar o vértice 6, sendo selecionado o vértice 7 que é o único adjacente ainda não visitado.



Passo	Visitado	Não-Visitado	Pilha
11	0, 1, 3, 4, 5, 6	7	DFS(6) = 0, 2, 7
10	0, 1, 3, 4, 5, 2	6, 7	DFS(2) = 0, 6, 7
9	0, 1, 3, 4, 5	1, 2, 3, 4, 5, 6, 7	DFS(0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          DFS(G, adjacente);
5      end
6  end
    
```

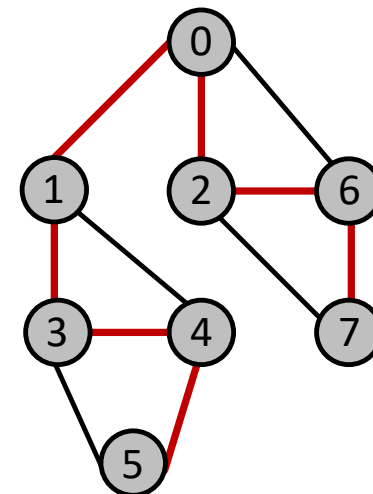
0 : [~~X~~, 2, 6]
 1 : [~~X~~, ~~X~~, ~~X~~]
 2 : [~~X~~, ~~X~~, ~~X~~]
 3 : [~~X~~, ~~X~~, ~~X~~]
 4 : [~~X~~, ~~X~~, ~~X~~]
 5 : [~~X~~, ~~X~~]
 6 : [~~X~~, ~~X~~, 7]
 ► 7 : [~~X~~, ~~X~~]

Quando a busca retorna a origem (vértice 0) é identificado um adjacente ainda não-visitado (vértice 2).

O processo de empilhamento recomeça a partir do vértice 2.

Continua ao empilhar o vértice 6, sendo selecionado o vértice 7 que é o único adjacente ainda não visitado.

No vértice 7 constata-se que todos seus adjacentes já foram visitados, iniciando o processo de **desempilhamento**.



Passo	Visitado	Não-Visitado	Pilha
12	0, 1, 3, 4, 5, 2, 6, 7	-	$DFS(7) = \textcolor{red}{2}, 6 $
11	0, 1, 3, 4, 5, 2, 6	7	$DFS(6) = \textcolor{red}{0}, 2, 7 $
10	0, 1, 3, 4, 5, 2	6, 7	$DFS(2) = \textcolor{red}{0}, 6, 7 $
9	0, 1, 3, 4, 5	1, 2, 3, 4, 5, 6, 7	$DFS(0) = \textcolor{red}{1}, 2, 6 $
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

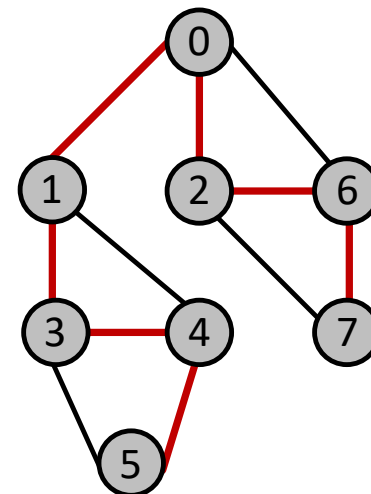
Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          | DFS(G, adjacente);
5      end
6  end
    
```

Vértices **6** e **2** são desempilhados,
pois todos seus adjacentes já
foram analisados.



```

0 : [X, 2, 6]
1 : [X, X, X]
2 : [X, X, X]
3 : [X, X, X]
4 : [X, X, X]
5 : [X, X]
▶ 6 : [X, X, X]
7 : [X, X]
    
```

Passo	Visitado	Não-Visitado	Pilha
13	0, 1, 3, 4, 5, 2, 6, 7	-	$DFS(6) = 0, 2, 7 $
10	0, 1, 3, 4, 5, 2	6, 7	$DFS(2) = 0, 6, 7 $
9	0, 1, 3, 4, 5	1, 2, 3, 4, 5, 6, 7	$DFS(0) = 1, 2, 6 $
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

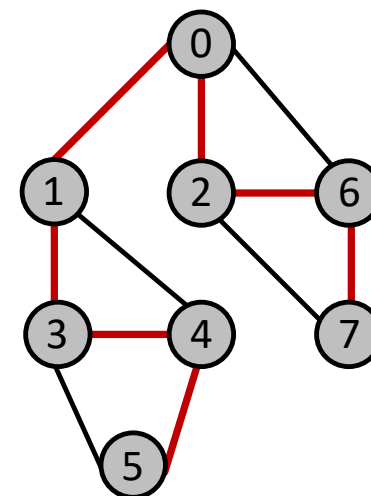
Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca v como visitado;
2  for cada adjacente de v do
3      if adjacente não-visitado then
4          | DFS(G, adjacente);
5      end
6  end
    
```

Vértices 6 e 2 são desempilhados,
pois todos seus adjacentes já
foram analisados.



0 : [~~X~~, 2, 6]
 1 : [~~X~~, ~~X~~, ~~X~~]
 ► 2 : [~~X~~, ~~X~~, ~~X~~]
 3 : [~~X~~, ~~X~~, ~~X~~]
 4 : [~~X~~, ~~X~~, ~~X~~]
 5 : [~~X~~, ~~X~~]
 6 : [~~X~~, ~~X~~, ~~X~~]
 7 : [~~X~~, ~~X~~]

Passo	Visitado	Não-Visitado	Pilha
14	0, 1, 3, 4, 5, 2, 6, 7	-	DFS(2) = 0, 6, 7
9	0, 1, 3, 4, 5	1, 2, 3, 4, 5, 6, 7	DFS(0) = 1, 2, 6
0	-	0, 1, 2, 3, 4, 5, 6, 7	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Recursivo)

DFS (G, v)

```

1  marca  $v$  como visitado;
2  for cada adjacente de  $v$  do
3      if adjacente não-visitado then
4          |   DFS( $G$ , adjacente);
5      end
6  end

```

```

0 : [X, X, X]
1 : [X, X, X]
2 : [X, X, X]
3 : [X, X, X]
4 : [X, X, X]
5 : [X, X]
6 : [X, X, X]
7 : [X, X]

```

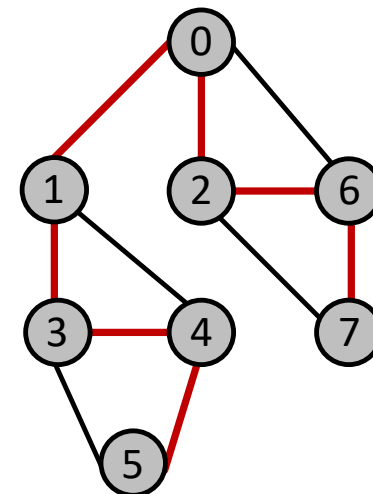
Vértices 6 e 2 são desempilhados,
pois todos seus adjacentes já
foram analisados.

A busca retorna ao vértice origem.

Como todos os adjacentes da
origem foram visitados e a pilha
está vazia, a busca é finalizada.

A sequência de visitas dos vértices na Busca em
Profundidade considerando o vértice 0 como origem é:

$DFS(0) = \{0, 1, 3, 4, 5, 2, 6, 7\}$



Passo	Visitado	Não-Visitado	Pilha
16	0, 1, 3, 4, 5, 2, 6, 7	-	



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Iterativo)

DFS(*G*, *v*)

```
1 naoVisitados ← [0, 1, ..., |E| - 1];  
2 visitados ← [];  
3 P ← [v];  
4 while naoVisitados ≠ ∅ do  
5   while P ≠ ∅ do  
6     t ← P[topo];  
7     if t ∉ visitados then  
8       | visitados ← visitados + t;  
9       adjViaveis ← listaAdj[t] \ visitados;  
10      if adjViaveis ≠ ∅ then  
11        | P ← adjViaveis[0]  
12      else  
13        | P.remove(topo);  
14      naoVisitados ← naoVisitados \ visitados;  
15      if naoVisitados ≠ ∅ then  
16        | P ← naoVisitados[0];  
17 return visitados;
```

No início, todos os vértices são considerados não visitados.

Inicializa a lista dos vértices visitados conforme a busca

Define a pilha *P* e insere o vértice inicial *v*.



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Iterativo)

DFS(*G*, *v*)

```
1 naoVisitados ← [0, 1, ..., |E| - 1];
2 visitados ← [ ];
3 P ← [v];
4 while naoVisitados ≠ ∅ do
5   while P ≠ ∅ do
6     t ← P[topo];
7     if t ∉ visitados then
8       | visitados ← visitados + t;
9       adjViaveis ← listaAdj[t] \ visitados;
10      if adjViaveis ≠ ∅ then
11        | P ← adjViaveis[0]
12      else
13        | P.remove(topo);
14      naoVisitados ← naoVisitados \ visitados;
15      if naoVisitados ≠ ∅ then
16        | P ← naoVisitados[0];
17 return visitados;
```

No início, todos os vértices são considerados não visitados.

Inicializa a lista dos vértices visitados conforme a busca

Define a pilha *P* e insere o vértice inicial *v*.

Enquanto houver vértices não visitados (linha 4) e

Enquanto houver vértices na pilha *P* (5) repete os passos:

Obtém o vértice *t* no topo de *P* (6); insere *t* na lista *visitados* caso ainda não esteja presente (7–8);



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Iterativo)

```
DFS( $G, v$ )  
1  $naoVisitados \leftarrow [0, 1, \dots, |E| - 1]$ ;  
2  $visitados \leftarrow []$ ;  
3  $P \leftarrow [v]$ ;  
4 while  $naoVisitados \neq \emptyset$  do  
5   while  $P \neq \emptyset$  do  
6      $t \leftarrow P[topo]$ ;  
7     if  $t \notin visitados$  then  
8        $visitados \leftarrow visitados + t$ ;  
9      $adjViaveis \leftarrow listaAdj[t] \setminus visitados$ ;  
10    if  $adjViaveis \neq \emptyset$  then  
11       $P \leftarrow adjViaveis[0]$   
12    else  
13       $P.remove(topo)$ ;  
14     $naoVisitados \leftarrow naoVisitados \setminus visitados$ ;  
15    if  $naoVisitados \neq \emptyset$  then  
16       $P \leftarrow naoVisitados[0]$ ;  
17 return  $visitados$ ;
```

No início, todos os vértices são considerados não visitados.

Inicializa a lista dos vértices visitados conforme a busca

Define a pilha P e insere o vértice inicial v .

Enquanto houver vértices não visitados (linha 4) e

Enquanto houver vértices na pilha P (5) repete os passos:

Obtém o vértice t no topo de P (6); insere t na lista $visitados$ caso ainda não esteja presente (7–8);

Em (9) obtém os adjacentes de t que ainda não foram incluídos na lista $visitados$; caso exista (10), insere o adjacente viável de menor id na pilha P (11); caso não exista, remove o vértice no topo de P (12–13).



2.7. Busca em Grafos – Busca em Profundidade

Algoritmo – DFS (Iterativo)

```
DFS( $G, v$ )  
1  $naoVisitados \leftarrow [0, 1, \dots, |E| - 1]$ ;  
2  $visitados \leftarrow []$ ;  
3  $P \leftarrow [v]$ ;  
4 while  $naoVisitados \neq \emptyset$  do  
5   while  $P \neq \emptyset$  do  
6      $t \leftarrow P[topo]$ ;  
7     if  $t \notin visitados$  then  
8        $visitados \leftarrow visitados + t$ ;  
9      $adjViaveis \leftarrow listaAdj[t] \setminus visitados$ ;  
10    if  $adjViaveis \neq \emptyset$  then  
11       $P \leftarrow adjViaveis[0]$   
12    else  
13       $P.remove(topo)$ ;  
14     $naoVisitados \leftarrow naoVisitados \setminus visitados$ ;  
15    if  $naoVisitados \neq \emptyset$  then  
16       $P \leftarrow naoVisitados[0]$ ;  
17 return  $visitados$ ;
```

No início, todos os vértices são considerados não visitados.

Inicializa a lista dos vértices visitados conforme a busca

Define a pilha P e insere o vértice inicial v .

Enquanto houver vértices não visitados (linha 4) e

Enquanto houver vértices na pilha P (5) repete os passos:

Obtém o vértice t no topo de P (6); insere t na lista $visitados$ caso ainda não esteja presente (7–8);

Em (9) obtém os adjacentes de t que ainda não foram incluídos na lista $visitados$; caso exista (10), insere o adjacente viável de menor id na pilha P (11); caso não exista, remove o vértice no topo de P (12–13).

Em (14) atualiza a lista $naoVisitados$, excluindo todos os vértices que constam como visitados.

Caso exista vértice não visitado, insere o de menor id na pilha P (15–16); ou retorna a sequência $visitados$ (17).



Análise de Complexidade

Busca em Largura (BFS)

Cada **vértice só entra na fila uma vez**.
Inserir e remover na fila possuem complexidade constante, as quais são realizadas $|V|$ vezes cada.

A lista de adjacências de cada vértice é examinada apenas uma vez, e a soma dos comprimentos de todas as listas é $(|E|)$.

Logo, se o grafo for representado por uma **lista de adjacências**, a complexidade $O(V + E)$.



Análise de Complexidade

Busca em Largura (BFS)

Cada vértice só entra na fila uma vez. Inserir e remover na fila possuem complexidade constante, as quais são realizadas $|V|$ vezes cada.

A lista de adjacências de cada vértice é examinada apenas uma vez, e a soma dos comprimentos de todas as listas é $(|E|)$.

Logo, se o grafo for representado por uma lista de adjacências, a complexidade $O(V + E)$.

Busca em Profundidade (DFS)

Para cada vértice do grafo a busca percorre todos os seus vizinhos.

Cada aresta é visitada duas vezes (empilhamento e desempilhamento).

Se o grafo for representado por uma lista de adjacências, a complexidade é $O(V + E)$.

Possui um maior custo de espaço devido a pilha recursiva.

Perguntas? Sugestões?

